



南開大學
Nankai University

《并行与分布式程序设计》实验报告

(2025~2026 学年第一学期)

实验名称: OpenMP 编程练习

姓 名: 李宗杭

学 号: 2311451

专 业: 软件工程

2025 年 12 月 9 日

目 录

1 梯形积分法的 Pthread 与 OpenMP 版本实现及对比	2
1.1 问题描述	2
1.2 算法设计与实现	2
1.2.1 串行方法	2
1.2.2 Pthread 线程函数-静态分配任务	2
1.2.3 Pthread 线程函数-动态分配任务	3
1.2.4 Pthread 主进程方法	4
1.2.5 OpenMP 静态调度	5
1.2.6 OpenMP 动态调度	5
1.2.7 OpenMP 引导调度	6
1.3 实验及结果分析	6
1.3.1 测试方法	6
1.3.2 结果分析	7
1.3.3 实际运行结果	10
2 ”多个数组排序”的 OpenMP 实现及循环调度方案对比	11
2.1 问题描述	11
2.2 算法设计与实现	11
2.2.1 数组初始化	11
2.2.2 静态调度	11
2.2.3 动态调度	12
2.2.4 引导调度	12
2.3 实验及结果分析	13
2.3.1 测试方法	13
2.3.2 结果分析	13
2.3.3 实际运行结果	17
3 高斯消去法的 OpenMP 实现	18
3.1 问题描述	18
3.2 算法设计与实现	18
3.2.1 静态调度	18
3.2.2 动态调度	19
3.2.3 引导调度	20
3.3 实验及结果分析	20
3.3.1 测试方法	20
3.3.2 结果分析	20
3.3.3 实际运行结果	25

1 梯形积分法的 Pthread 与 OpenMP 版本实现及对比

1.1 问题描述

分别实现课件中的梯形积分法的 Pthread、OpenMP 版本，熟悉并掌握 OpenMP 编程方法，探讨两种编程方式的异同。

1.2 算法设计与实现

1.2.1 串行方法

串行实现的方法，用于作为对照。

```
1 // 串行梯形积分法
2 double trap_serial(double a, double b, int n) {
3     double h = (b - a) / n;
4     double approx = (f(a) + f(b)) / 2.0;
5     for (int i = 1; i <= n - 1; i++) {
6         double x = a + i * h;
7         approx += f(x);
8     }
9     return approx * h;
10 }
```

1.2.2 Pthread 线程函数-静态分配任务

将全部任务一次性均分给各个线程。

```
1 // Pthread 线程函数-静态分配任务
2 void* trap_pthread_thread_static(void* param) {
3     PthreadTrapParam* p = (PthreadTrapParam*)param;
4     double a = p->a;
5     double h = p->h;
6     int n = p->n;
7     int thread_id = p->thread_id;
8     int thread_count = p->thread_count;
9
10    // 划分当前线程的局部任务
11    int local_n = (n + thread_count - 1) / thread_count;
12    int left = thread_id * local_n + 1;
13    int right = min(left + local_n - 1, n - 1);
14
15    // 局部积分计算
16    double local_result = 0.0;
17    for (int i = left; i <= right; i++) {
```

```
18     double x = a + i * h;
19     local_result += f(x);
20 }
21
22 // 记录局部结果
23 p->local_result = local_result;
24
25 pthread_exit(NULL);
26 return NULL;
27 }
```

后续动态分配任务和粗粒度动态的测试函数与 test_static() 基本一致，仅多了互斥量的初始化与销毁以及创建线程传递的线程函数不一样，不再重复展示。

1.2.3 Pthread 线程函数-动态分配任务

各个线程通过竞争互斥锁获取将要处理的任务，其中 chunk 为每次分配的任务大小，可手动调整。

```
1 // Pthread 线程函数-动态分配任务
2 void* trap_pthread_thread_dynamic(void* param) {
3     PthreadTrapParam* p = (PthreadTrapParam*)param;
4     double a = p->a;
5     double h = p->h;
6     int n = p->n;
7     int chunk = p->chunk;
8     int thread_id = p->thread_id;
9     int thread_count = p->thread_count;
10
11     int task;
12     double local_result = 0.0;
13
14     while (true) {
15         // 动态获取任务块
16         pthread_mutex_lock(&task_mutex);
17         task = next_task;
18         next_task += chunk;
19         pthread_mutex_unlock(&task_mutex);
20         if (task >= n) {
21             break;
22         }
23
24         // 划分当前线程的局部任务
25         int local_n = chunk;
```

```
26     int left = task;
27     int right = min(left + local_n - 1, n - 1);
28
29     // 局部积分计算
30     for (int i = left; i <= right; i++) {
31         double x = a + i * h;
32         local_result += f(x);
33     }
34 }
35
36 // 记录局部结果
37 p->local_result = local_result;
38
39 pthread_exit(NULL);
40 return NULL;
41 }
```

1.2.4 Pthread 主进程方法

仅一次性创建所有线程，通过传入的 chunk 参数选择执行静态分配任务或动态分配任务。在此将待积分函数区间首尾处理好，线程函数仅处理中间部分。

```
1 // Pthread 并行梯形积分法
2 double trap_pthread(double a, double b, int n, int thread_count, int chunk) {
3     vector<pthread_t> threads(thread_count);
4     vector<PthreadTrapParam> params(thread_count);
5     pthread_mutex_init(&task_mutex, NULL);
6     double global_result = (f(a) + f(b)) / 2.0; // 全局积分结果
7     double h = (b - a) / n;
8     next_task = 1;
9
10    // 初始化线程参数，创建线程
11    for (int i = 0; i < thread_count; i++) {
12        params[i].a = a;
13        params[i].h = h;
14        params[i].n = n;
15        params[i].chunk = chunk;
16        params[i].thread_id = i;
17        params[i].thread_count = thread_count;
18        if (chunk == 0) {
19            pthread_create(&threads[i], NULL, trap_pthread_thread_static,
20                          ↪ &params[i]);
21        }
22    }
23 }
```

```

20     }
21     else {
22         pthread_create(&threads[i], NULL, trap_pthread_thread_dynamic,
23             ↪ &params[i]);
24     }
25
26     // 等待所有线程完成, 汇总局部结果
27     for (int i = 0; i < thread_count; i++) {
28         pthread_join(threads[i], NULL);
29         global_result += params[i].local_result;
30     }
31
32     pthread_mutex_destroy(&task_mutex);
33     return global_result * h;
34 }

```

1.2.5 OpenMP 静态调度

pragma omp parallel for: 并行化后续的 for 循环;

num_threads(thread_count): 指定并行使用的线程数;

schedule(static): 静态调度策略, 将 for 循环的迭代任务提前平均分配给各个线程;

reduction(+:approx): 归约操作, 解决多线程对共享变量 approx 的竞争问题。

```

1  // OpenMP 并行梯形积分法-静态调度
2  double trap_omp_static(double a, double b, int n, int thread_count) {
3      double h = (b - a) / n;
4      double approx = (f(a) + f(b)) / 2.0;
5
6      #pragma omp parallel for num_threads(thread_count) schedule(static)
7      ↪ reduction(+:approx)
8      for (int i = 1; i <= n - 1; i++) {
9          double x = a + i * h;
10         approx += f(x);
11     }
12
13     return approx * h;
14 }

```

1.2.6 OpenMP 动态调度

schedule(dynamic, chunk): 动态调度策略, 迭代被划分为大小为 chunk 的块, 线程完成一个块后再获取下一个块。

```
1 // OpenMP 并行梯形积分法-动态调度
2 double trap_omp_dynamic(double a, double b, int n, int thread_count, int chunk)
3 ↪ {
4     double h = (b - a) / n;
5     double approx = (f(a) + f(b)) / 2.0;
6
7     #pragma omp parallel for num_threads(thread_count) schedule(dynamic, chunk)
8     ↪ reduction(+:approx)
9     for (int i = 1; i <= n - 1; i++) {
10         double x = a + i * h;
11         approx += f(x);
12     }
13
14     return approx * h;
15 }
```

1.2.7 OpenMP 引导调度

schedule(guided): 引导调度策略, 块大小随时间递减, 初始块较大, 后续逐渐变小。

```
1 // OpenMP 并行梯形积分法-引导调度
2 double trap_omp_guided(double a, double b, int n, int thread_count) {
3     double h = (b - a) / n;
4     double approx = (f(a) + f(b)) / 2.0;
5
6     #pragma omp parallel for num_threads(thread_count) schedule(guided)
7     ↪ reduction(+:approx)
8     for (int i = 1; i <= n - 1; i++) {
9         double x = a + i * h;
10         approx += f(x);
11     }
12
13     return approx * h;
14 }
```

1.3 实验及结果分析

1.3.1 测试方法

测试不同线程数及不同分块大小下各方法的运行情况。

- 被积分函数 $f(x)$: $x^2 + 2x + 1$;

- 积分区间 $[a, b]$: $[0, 10]$;
- 划分粒度 n : 1,000,000。

每种情况每个方法测试五次取平均值，获取稳定结果，降低误差。

1.3.2 结果分析

(1) 不同线程数下各个方法的运行时间对比

先看图1.1，可以看到由于为两个动态算法设置的分块大小为 1，因此两个动态算法的耗时远高于其他算法，甚至远高于串行方法。此外当线程数较小时，随着线程数的增加，两种动态方式耗时也增加，不断将任务分配的锁竞争和线程切换所带来的开销放大。而当线程数增大到一定程度后，pthread 动态方法的耗时依旧在缓步提升，但 omp 动态调度的耗时却开始下降，这是因为 OpenMP 动态调度是编译器优化后的高层实现，能通过锁粒度、任务分配的自适应调整，让开销随线程数增大而收敛，比如将“全局任务池锁”拆分为更细的局部锁，减少线程抢任务的冲突。

再来看图1.2，删去了两条耗时过高的动态方法的折线，以此来更细致的对比其他方法。首先很明显，所有合理的并行方法均优于串行方法。随着线程数的增大，omp 静态调度和 omp 引导调度耗时均逐渐下降，且下降速度逐步放缓，且两种方式耗时始终不相上下，因为梯形积分法的各部分计算量相当，几乎不会出现负载不均的情况，因此静态调度的效果也十分好。而 pthread 静态分配任务虽然也在线程数增长到 8 时耗时大幅下降，但后续随着线程数继续增大，耗时却又逐步回升，推测是因为如表1所示，梯形积分法计算量很小，大部分情况下平均运行时间不超过 1ms，因此当线程数变大时创建线程所产生的开销会超过任务本身的开销，而 omp 则由于编译器的优化而不会出现这种情况。

线程数	串行	pthread 静态	pthread 动态	omp 静态	omp 动态	omp 引导
2	1.5943	1.019	27.497	0.8829	26.325	0.778
4	1.5943	1.1545	28.224	0.45632	26.616	0.48597
6	1.5943	16.05	37.947	0.30943	31.033	0.33404
8	1.5943	0.55278	39.63	0.24811	36.912	0.2337
10	1.5943	0.63556	40.67	0.34811	35.054	0.23428
12	1.5943	0.55907	41.252	0.30735	32.265	0.21272
14	1.5943	0.69309	41.105	0.28202	35.963	0.21204
16	1.5943	0.85693	41.145	0.26716	32.364	0.20915
18	1.5943	0.79954	42.142	0.24338	35.241	0.22644
20	1.5943	0.88482	41.954	0.23348	36.587	0.20804
22	1.5943	0.92113	43.575	0.23604	35.5	0.22792
24	1.5943	1.0167	41.646	0.22145	31.888	0.19662
26	1.5943	1.0842	42.566	0.21751	29.883	0.20549
28	1.5943	1.129	41.948	0.22182	25.505	0.20923
30	1.5943	1.2185	42.002	0.20186	23.991	0.21046
32	1.5943	1.2795	42.626	0.19464	23.436	0.2156

表 1: 不同线程数下各算法运行时间（单位：ms）

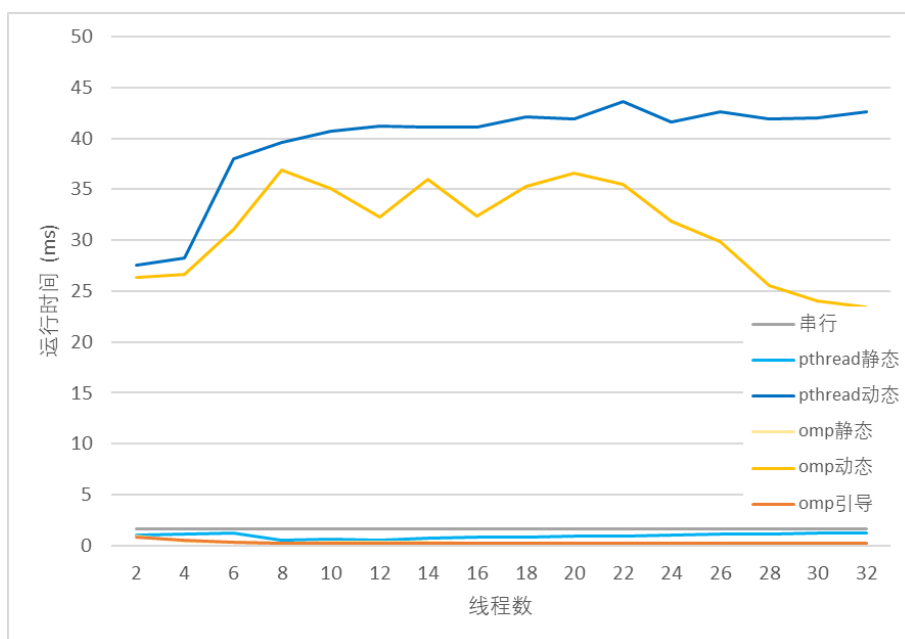


图 1.1: 不同线程数下各个方法的运行时间（动态分块大小: 1）

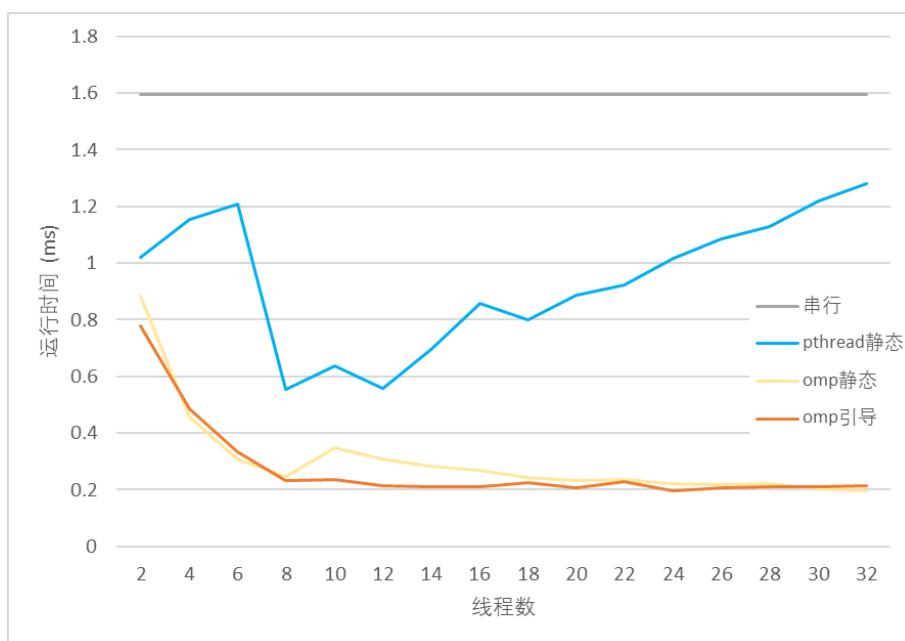


图 1.2: 不同线程数下各个方法的运行时间（除去动态）

(2) 不同分块大小下各个方法的运行时间对比

在这次实验中分块大小只会影响 pthread 动态分配任务方法和 omp 动态调度方法（并未给 omp 静态调度和 omp 引导调度设置 chunk 参数）。

如图1.3，随着分块越来越大，即每次动态分配的任务量越来越大，动态算法的耗时越来越低，这是由于单次分配的任务量大，而任务总量不变，则分配次数减少，线程切换和锁竞争的开销就会减少。并且当分块大小达到本次测试的最大值 (4096) 时，pthread 动态分配的效率超过了 pthread 静态分配，omp 动态调度超过了 omp 静态调度，说明静态分配还是存在一定的负载不均。

动态分块	串行	pthread 静态	pthread 动态	omp 静态	omp 动态	omp 引导
1	1.5943	0.77735	42.799	0.26366	31.37	0.20958
2	1.5943	0.72857	22.279	0.24692	17.325	0.22166
4	1.5943	0.67389	13.525	0.24682	9.1223	0.22233
8	1.5943	0.69373	9.3074	0.24628	4.5975	0.21453
16	1.5943	0.67793	6.8484	0.24737	2.3638	0.20204
32	1.5943	0.64871	4.3046	0.26237	1.201	0.20081
64	1.5943	0.66889	2.4189	0.24574	0.62559	0.19982
128	1.5943	0.68261	1.3934	0.24804	0.33574	0.19721
256	1.5943	0.65345	0.81857	0.25017	0.20999	0.19589
512	1.5943	0.70251	0.69633	0.41251	0.2058	0.22005
1024	1.5943	0.71511	0.68991	0.35093	0.19525	0.20505
2048	1.5943	0.60895	0.63259	0.25428	0.19241	0.20074
4096	1.5943	0.74032	0.72573	0.35817	0.19692	0.20498

表 2: 不同动态分块大小下各算法运行时间 (单位: ms)

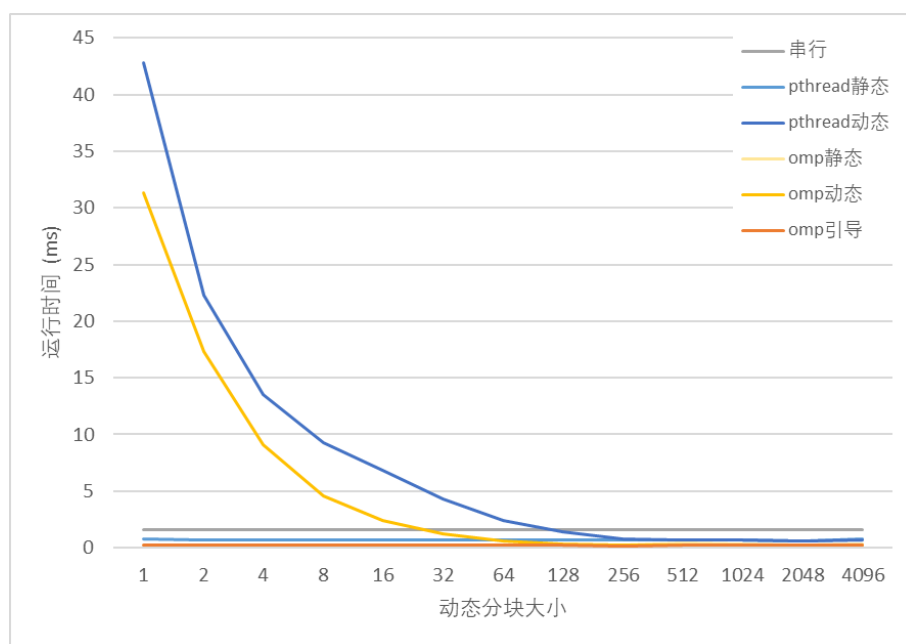


图 1.3: 不同动态分块大小下各个方法运行时间

(3) 两种编程方式的异同

相同点

1. 均基于共享内存模型, 支持多线程并行计算, 通过任务划分实现梯形积分的并行化。
2. 都可实现静态和动态调度方式, 核心是将积分区间分割后分配给不同线程。
3. 需处理线程同步问题 (如 Pthread 的互斥锁、OpenMP 的归约子句), 确保全局结果正确。

不同点

对比维度	Pthread	OpenMP
编程复杂度	底层 API 调用, 需手动创建线程、管理参数、实现同步, 代码量大	编译指示简化编程, 无需手动管理线程, 编译器自动处理任务划分和同步
调度灵活性	静态/动态调度均需手动实现 (如动态调度的任务池、互斥锁控制), 灵活但繁琐	内置 static/dynamic/guided 三种调度, 通过 schedule 子句直接配置, 无需手动编码
性能开销	动态调度中互斥锁竞争导致高开销, 线程创建/销毁成本高	编译器优化充分, 归约子句、调度策略效率高, 静态/引导调度几乎无额外开销
开发效率	低, 需关注线程管理、同步细节, 调试难度大	高, 串行代码基础上添加少量编译指示即可并行化, 支持增量开发
可移植性	依赖 POSIX 标准, 跨平台需适配 (如 Windows 与 Linux 的线程 API 差异)	编译器原生支持, 跨平台兼容性更好

1.3.3 实际运行结果

线程数	串行	pthread静态	pthread动态	omp静态	omp动态	omp引导
2	1.5943	1.019	27.497	0.8829	26.325	0.778
4	1.5943	1.1545	28.224	0.45632	26.616	0.48597
6	1.5943	16.05	37.947	0.30943	31.033	0.33404
8	1.5943	0.55278	39.63	0.24811	36.912	0.2337
10	1.5943	0.63556	40.67	0.34811	35.054	0.23428
12	1.5943	0.55907	41.252	0.30735	32.265	0.21272
14	1.5943	0.69309	41.105	0.28202	35.963	0.21204
16	1.5943	0.85693	41.145	0.26716	32.364	0.20915
18	1.5943	0.79954	42.142	0.24338	35.241	0.22644
20	1.5943	0.88482	41.954	0.23348	36.587	0.20804
22	1.5943	0.92113	43.575	0.23604	35.5	0.22792
24	1.5943	1.0167	41.646	0.22145	31.888	0.19662
26	1.5943	1.0842	42.566	0.21751	29.883	0.20549
28	1.5943	1.129	41.948	0.22182	25.505	0.20923
30	1.5943	1.2185	42.002	0.20186	23.991	0.21046
32	1.5943	1.2795	42.626	0.19464	23.436	0.2156
动态分块	串行	pthread静态	pthread动态	omp静态	omp动态	omp引导
1	1.5943	0.77735	42.799	0.26366	31.37	0.20958
2	1.5943	0.72857	22.279	0.24692	17.325	0.22166
4	1.5943	0.67389	13.525	0.24682	9.1223	0.22233
8	1.5943	0.69373	9.3074	0.24628	4.5975	0.21453
16	1.5943	0.67793	6.8484	0.24737	2.3638	0.20204
32	1.5943	0.64871	4.3046	0.26237	1.201	0.20081
64	1.5943	0.66889	2.4189	0.24574	0.62559	0.19982
128	1.5943	0.68261	1.3934	0.24804	0.33574	0.19721
256	1.5943	0.65345	0.81857	0.25017	0.20999	0.19589
512	1.5943	0.70251	0.69633	0.41251	0.2058	0.22005
1024	1.5943	0.71511	0.68991	0.35093	0.19525	0.20505
2048	1.5943	0.60895	0.63259	0.25428	0.19241	0.20074
4096	1.5943	0.74032	0.72573	0.35817	0.19692	0.20498

其中 (线程数 6, pthread 静态) 这个值 (16.05), 显著高于周围其他值, 经多次测试为系统误差所致, 在结果分析时, 根据多次测试结果手动为其赋予了一个合理的值。

2 “多个数组排序”的 OpenMP 实现及循环调度方案对比

2.1 问题描述

对于课件中“多个数组排序”的任务不均衡案例进行 OpenMP 编程实现（规模可自己调整），并探索不同循环调度方案的优劣。可从任务分块的大小、线程数的多少、静态动态多线程结合等方面进行尝试，探索规律。

2.2 算法设计与实现

2.2.1 数组初始化

模拟负载不均衡。

```
1 // 初始化数组（模拟负载不均衡）
2 void init_arrays() {
3     arrays.resize(arr_num);
4     srand(unsigned(time(NULL)));
5     int seg = arr_num / 4; // 分为 4 段，有序度依次降低
6
7     for (int i = 0; i < arr_num; i++) {
8         arrays[i].resize(arr_len);
9         int ratio;
10        if (i < seg) ratio = 0; // 完全升序
11        else if (i < 2 * seg) ratio = 32; // 1/4 逆序
12        else if (i < 3 * seg) ratio = 64; // 1/2 逆序
13        else ratio = 128; // 完全逆序
14
15        if ((rand() & 127) < ratio) {
16            for (int j = 0; j < arr_len; j++) arrays[i][j] = j;
17        } else {
18            for (int j = 0; j < arr_len; j++) arrays[i][j] = arr_len - j;
19        }
20    }
21 }
```

2.2.2 静态调度

其中线程数和分块大小可控。

```
1 // OpenMP 静态调度排序
2 void sort_omp_static(int thread_count, int chunk) {
3     if (chunk == 0) {
4         #pragma omp parallel for num_threads(thread_count) schedule(static)
5         for (int i = 0; i < arr_num; i++) {
```

```

6         stable_sort(arrays_copy[i].begin(), arrays_copy[i].end());
7     }
8 }
9 else {
10     #pragma omp parallel for num_threads(thread_count) schedule(static,
    ↪ chunk)
11     for (int i = 0; i < arr_num; i++) {
12         stable_sort(arrays_copy[i].begin(), arrays_copy[i].end());
13     }
14 }
15 }

```

2.2.3 动态调度

```

1  // OpenMP 动态调度排序
2  void sort_omp_dynamic(int thread_count, int chunk) {
3      if (chunk == 0) {
4          #pragma omp parallel for num_threads(thread_count) schedule(dynamic)
5          for (int i = 0; i < arr_num; i++) {
6              stable_sort(arrays_copy[i].begin(), arrays_copy[i].end());
7          }
8      }
9      else {
10         #pragma omp parallel for num_threads(thread_count) schedule(dynamic,
    ↪ chunk)
11         for (int i = 0; i < arr_num; i++) {
12             stable_sort(arrays_copy[i].begin(), arrays_copy[i].end());
13         }
14     }
15 }

```

2.2.4 引导调度

```

1  // OpenMP 引导调度排序
2  void sort_omp_guided(int thread_count, int chunk) {
3      if (chunk == 0) {
4          #pragma omp parallel for num_threads(thread_count) schedule(guided)
5          for (int i = 0; i < arr_num; i++) {
6              stable_sort(arrays_copy[i].begin(), arrays_copy[i].end());
7          }
8      }
9      else {

```

```

10         #pragma omp parallel for num_threads(thread_count) schedule(guided,
        ↪ chunk)
11         for (int i = 0; i < arr_num; i++) {
12             stable_sort(arrays_copy[i].begin(), arrays_copy[i].end());
13         }
14     }
15 }

```

2.3 实验及结果分析

2.3.1 测试方法

首先测试各个方法在不同线程数下的运行情况，取最优的线程数再测试不同分块大小的运行情况，最后选取一个较优分块测试在不同数据规模下的情况。

2.3.2 结果分析

(1) 不同线程数下各个方法的运行时间对比

如图2.4，三种调度方式耗时均随线程数增加而下降，虽下降速率逐步放缓，但直到本次测试最大线程数 (32) 时仍在下降，说明 32 线程未达到“线程管理开销大于并行收益”的临界点，是当前数据规模下的较优线程数。

结合表3再进行细致的分析，动态调度始终优于另外两种策略，测试数据刻意模拟负载不均衡的场景，而动态调度通过其策略特点实现了负载均衡。引导调度也实现了负载均衡，略优于静态调度，但其分块动态调整的复杂策略在此处反而不如单纯的动态调度。

线程数	串行	静态	动态	引导
2	5101.9	2724.2	2480.5	2699.1
4	5101.9	1450.3	1326.8	1461.7
6	5101.9	1005.5	919.68	1014.2
8	5101.9	784.58	809.58	907.76
10	5101.9	798.38	728.63	774.04
12	5101.9	706.68	647.77	690.31
14	5101.9	640.65	588.88	623.42
16	5101.9	593.84	542.45	572.85
18	5101.9	560.12	508.54	537.78
20	5101.9	522.97	483.37	512.98
22	5101.9	506.72	467.73	501.21
24	5101.9	488.19	454.35	482.03
26	5101.9	478.12	436.5	471.54
28	5101.9	460.38	449.32	459.13
30	5101.9	453.28	424.12	445.28
32	5101.9	441.39	399.07	442.45

表 3: 不同线程数下各个方法的运行时间 (单位: ms)

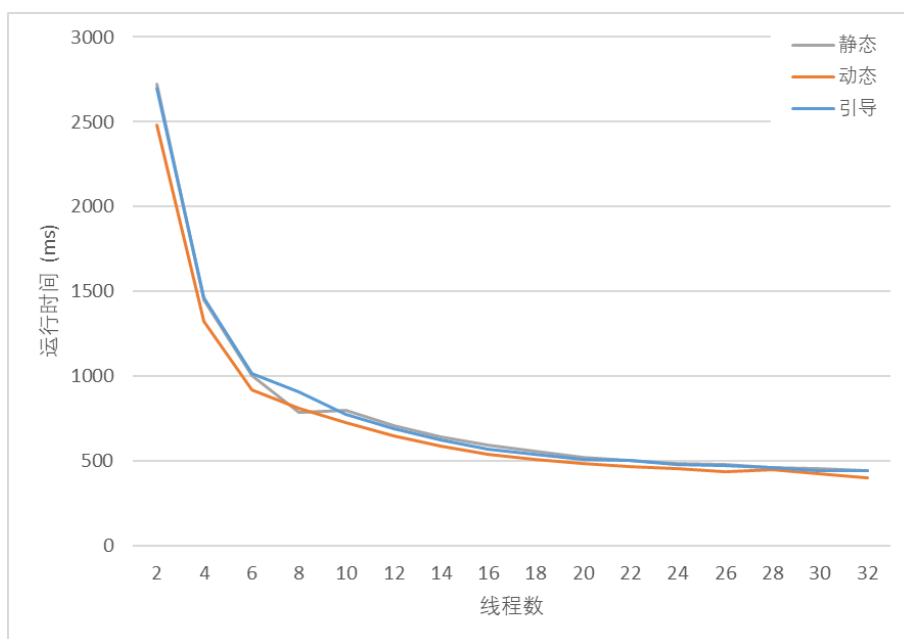


图 2.4: 不同线程数下各个方法的运行时间

(2) 不同分块大小下各个方法的运行时间对比

分块大小为 0 代表未手动设置 chunk 参数。

如图2.5, 在分块大小在 [1, 16] 这个区间内时, 三种策略的耗时均保持稳定, 且动态调度依旧最优。然而当分块变得更大时, 所有策略的耗时均增加, 测试数据规模仅 10000, 但分块大小却达到了数千, 并且数据规模并不能被分块整除, 很明显, 这导致了负载不均。并且测试采用了 32 线程, 在分块较小时, 所有线程均可投入使用, 当分块过大时, 会有许多线程闲置, 尤其是极端情况下 (4096), 有两个线程各自处理四千多个数组, 一个线程处理约两千个数组, 已经导致了负载不均, 并且还有 29 个线程处于了闲置, 性能得不到充分利用。

因此, 分块大小应设在合理的范围内, 不能为了减小线程切换和锁竞争的开销而过度增大分块大小。

分块大小	串行	静态	动态	引导
0	5101.9	440.41	400.36	437.9
1	5101.9	424.69	401.82	438.1
2	5101.9	425.19	399.91	434.54
4	5101.9	420.93	401.3	439.07
8	5101.9	425.74	402.73	432.48
16	5101.9	429.35	405.62	436.78
32	5101.9	429.48	408.59	447.88
64	5101.9	441.42	427.76	443.27
128	5101.9	474.38	440.03	448.04
256	5101.9	524.17	494.29	487.75
512	5101.9	581.48	572.69	569.23
1024	5101.9	823.94	813.55	799
2048	5101.9	1282.4	1277.7	1314
4096	5101.9	2308.7	2284.2	2295.9

表 4: 不同分块大小下各个方法的运行时间 (单位: ms)

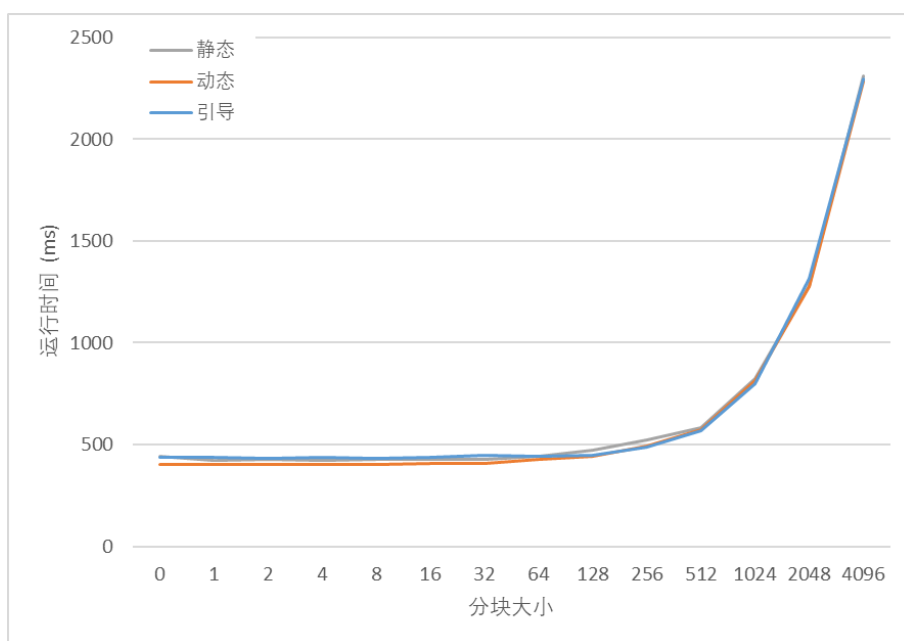


图 2.5: 不同分块大小下各个方法的运行时间

(3) 不同数据规模下各个方法的运行时间对比

如图2.6, 随着数据规模增大, 三种策略耗时均近似线性增长, 说明三种调度均能有效应对数据规模扩大, 并行架构可扩展。

静态调度与动态调度相差不大, 而引导调度与耗时略微变大, 与另两个差距逐渐变大, 或许是分块调整逻辑随数据规模扩大而更复杂。

数据规模	串行	静态	动态	引导
5000	2492.1	198.33	169.57	183.5
10000	4927.5	391.68	342.82	374.51
15000	7438.3	578.61	520.66	687.35
20000	9931.2	770.69	713.48	906.89
25000	12511	950.89	899.55	1141.4
30000	15001	1135.1	1106.7	1359
35000	17616	1322.5	1306.9	1575.2
40000	20369	1533	1560.8	1815.1
45000	22849	1722.2	1791.6	2054.6
50000	25087	1895.8	1964.7	2261.5

表 5: 不同数据规模下各个方法的运行时间 (单位: ms)

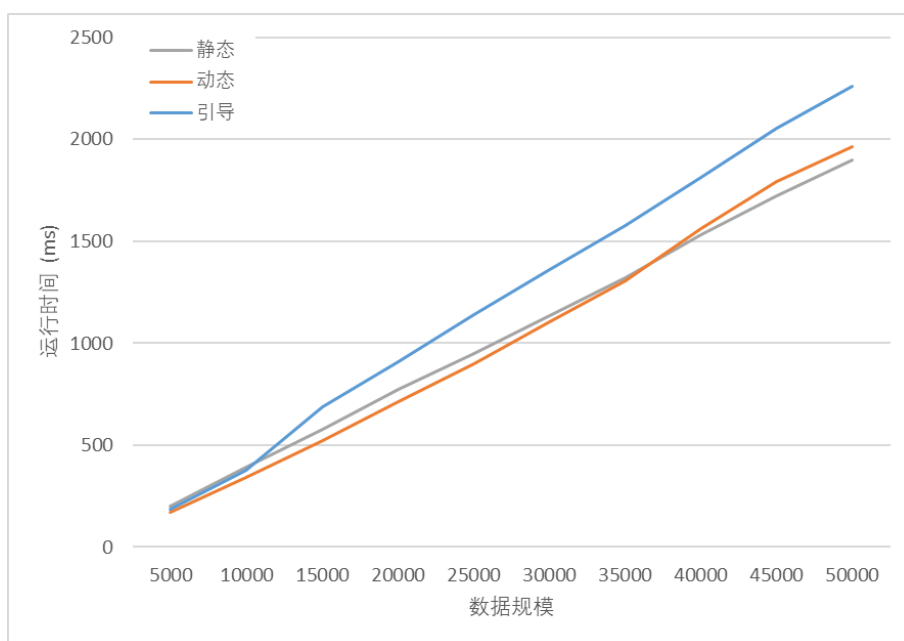


图 2.6: 不同数据规模下各个方法的运行时间

2.3.3 实际运行结果

数据规模: 10000 chunk: 无				
线程数	串行	静态	动态	引导
2	5101.9	2724.2	2480.5	2699.1
4	5101.9	1450.3	1326.8	1461.7
6	5101.9	1005.5	919.68	1014.2
8	5101.9	784.58	809.58	907.76
10	5101.9	798.38	728.63	774.04
12	5101.9	706.68	647.77	690.31
14	5101.9	640.65	588.88	623.42
16	5101.9	593.84	542.45	572.85
18	5101.9	560.12	508.54	537.78
20	5101.9	522.97	483.37	512.98
22	5101.9	506.72	467.73	501.21
24	5101.9	488.19	454.35	482.03
26	5101.9	478.12	436.5	471.54
28	5101.9	460.38	449.32	459.13
30	5101.9	453.28	424.12	445.28
32	5101.9	441.39	399.07	442.45
数据规模: 10000 线程数: 32				
分块大小	串行	静态	动态	引导
0	5101.9	440.41	400.36	437.9
1	5101.9	424.69	401.82	438.1
2	5101.9	425.19	399.91	434.54
4	5101.9	420.93	401.3	439.07
8	5101.9	425.74	402.73	432.48
16	5101.9	429.35	405.62	436.78
32	5101.9	429.48	408.59	447.88
64	5101.9	441.42	427.76	443.27
128	5101.9	474.38	440.03	448.04
256	5101.9	524.17	494.29	487.75
512	5101.9	581.48	572.69	569.23
1024	5101.9	823.94	813.55	799
2048	5101.9	1282.4	1277.7	1314
4096	5101.9	2308.7	2284.2	2295.9
chunk: 0 线程数: 32				
数据规模	串行	静态	动态	引导
5000	2492.1	198.33	169.57	183.5
10000	4927.5	391.68	342.82	374.51
15000	7438.3	578.61	520.66	687.35
20000	9931.2	770.69	713.48	906.89
25000	12511	950.89	899.55	1141.4
30000	15001	1135.1	1106.7	1359
35000	17616	1322.5	1306.9	1575.2
40000	20369	1533	1560.8	1815.1
45000	22849	1722.2	1791.6	2054.6
50000	25087	1895.8	1964.7	2261.5

图 2.7: ”多个数组排序”的 OpenMP 实现测试结果

3 高斯消去法的 OpenMP 实现

3.1 问题描述

实现高斯消去法解线性方程组的 OpenMP 编程，与 SSE/AVX 编程结合，并探索优化任务分配方法。

3.2 算法设计与实现

分别实现了三种调度方式，皆与 AVX 相结合，并实现串行 AVX 作为对照。

3.2.1 静态调度

静态调度不对 chunk 参数做调整。

```

1  // OpenMP 静态调度
2  void gauss_omp_static_avx(int n, int thread_count, int chunk) {
3      #pragma omp parallel num_threads(thread_count)
4      {
5          for (int k = 0; k < n - 1; k++) {
6              #pragma omp for schedule(static)
7              for (int i = k + 1; i < n; i++) {
8                  float factor = A[i][k] / A[k][k];
9                  int j = k;
10                 __m256 vec_factor = _mm256_set1_ps(factor);
11
12                 for (; j <= n - 8; j += 8) {
13                     __m256 vec_Ak = _mm256_loadu_ps(A[k] + j);
14                     __m256 vec_Ai = _mm256_loadu_ps(A[i] + j);
15                     vec_Ai = _mm256_sub_ps(vec_Ai, _mm256_mul_ps(vec_factor,
16                                     ↪ vec_Ak));
17                     _mm256_storeu_ps(A[i] + j, vec_Ai);
18                 }
19
20                 for (; j < n; j++) {
21                     A[i][j] -= factor * A[k][j];
22                 }
23                 B[i] -= factor * B[k];
24             }
25         }
26     }

```

3.2.2 动态调度

动态调度可选择使用默认 chunk 或手动设置 chunk。

```

1 // OpenMP 动态调度
2 void gauss_omp_dynamic_avx(int n, int thread_count, int chunk) {
3     #pragma omp parallel num_threads(thread_count)
4     {
5         for (int k = 0; k < n - 1; k++) {
6             if (chunk == 0) {
7                 #pragma omp for schedule(dynamic)
8                 for (int i = k + 1; i < n; i++) {
9                     float factor = A[i][k] / A[k][k];
10                    int j = k;
11                    __m256 vec_factor = _mm256_set1_ps(factor);
12
13                    for (; j <= n - 8; j += 8) {
14                        __m256 vec_Ak = _mm256_loadu_ps(A[k] + j);
15                        __m256 vec_Ai = _mm256_loadu_ps(A[i] + j);
16                        vec_Ai = _mm256_sub_ps(vec_Ai, _mm256_mul_ps(vec_factor,
17                            ↪ vec_Ak));
18                        _mm256_storeu_ps(A[i] + j, vec_Ai);
19                    }
20
21                    for (; j < n; j++) {
22                        A[i][j] -= factor * A[k][j];
23                    }
24                    B[i] -= factor * B[k];
25                }
26            } else {
27                #pragma omp for schedule(dynamic, chunk)
28                for (int i = k + 1; i < n; i++) {
29                    float factor = A[i][k] / A[k][k];
30                    int j = k;
31                    __m256 vec_factor = _mm256_set1_ps(factor);
32
33                    for (; j <= n - 8; j += 8) {
34                        __m256 vec_Ak = _mm256_loadu_ps(A[k] + j);
35                        __m256 vec_Ai = _mm256_loadu_ps(A[i] + j);
36                        vec_Ai = _mm256_sub_ps(vec_Ai, _mm256_mul_ps(vec_factor,
37                            ↪ vec_Ak));
38                        _mm256_storeu_ps(A[i] + j, vec_Ai);

```

```
38         }
39
40         for (; j < n; j++) {
41             A[i][j] -= factor * A[k][j];
42         }
43         B[i] -= factor * B[k];
44     }
45 }
46 }
47 }
48 }
```

3.2.3 引导调度

引导调度与动态调度仅为 `schedule(dynamic)` 与 `schedule(guided)` 的区别，在此不多做展示。

3.3 实验及结果分析

3.3.1 测试方法

首先测试各个方法在不同线程数下的运行情况，取最优的线程数再测试不同分块大小的运行情况，最后选取一个较优分块测试在不同数据规模下的情况。

3.3.2 结果分析

(1) 不同线程数下各个方法的运行时间对比

如图3.8，并行方法皆优于串行方法，随着线程数的增大，各 OpenMP 调度方式的耗时逐渐减小，在 8 线程时达到最低，而后又逐渐增大，甚至在达到 32 线程时超过了串行方法的耗时。

当线程数较小时，增加线程可以提高并行效率，并可以有效缓解负载不均，可以看到此次测试中 8 线程是最优的情况，但如果线程数过多的话，线程切换和锁竞争的开销会变大，甚至掩盖并行编程的收益。

线程数	串行	静态	动态	引导
2	127.87	75.971	88.541	73.72
4	127.87	48.975	54.539	45.684
6	127.87	53.034	45.349	38.492
8	127.87	48.887	43.139	37.566
10	127.87	48.55	53.585	40.801
12	127.87	57.127	57.416	45.395
14	127.87	55.455	71.486	66.063
16	127.87	60.001	78.574	73.432
18	127.87	65.181	90.522	82.666
20	127.87	73.245	91.233	86.712
22	127.87	81.999	100.45	93.318
24	127.87	91.38	106.42	101.67
26	127.87	96.509	114.65	109.29
28	127.87	106.98	115.13	111.1
30	127.87	113.61	122.79	116.62
32	127.87	114.15	130.07	125.9

表 6: 不同线程数下各方法运行时间 (单位: ms)

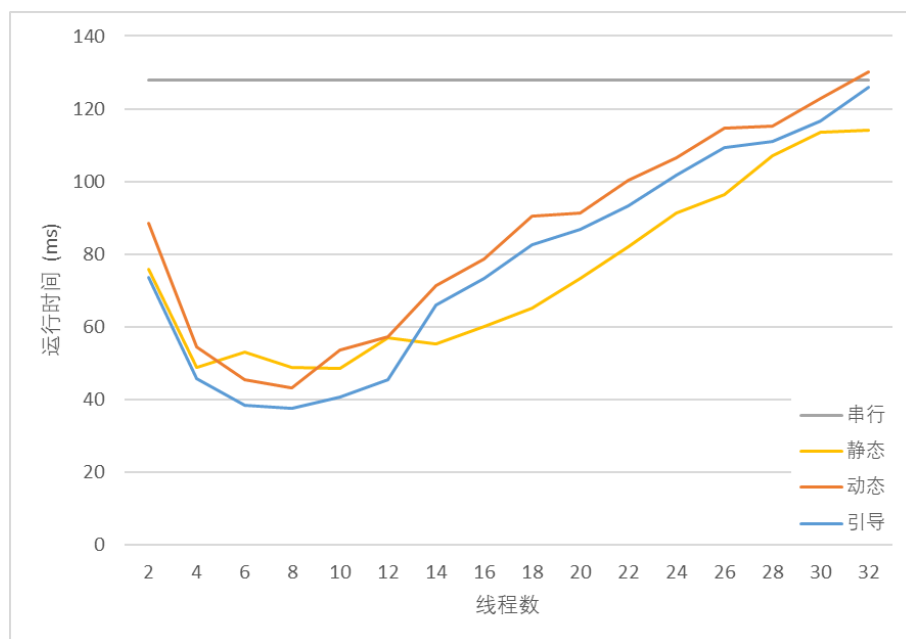


图 3.8: 不同线程数下各方法运行时间

(2) 不同分块大小下各个方法的运行时间对比

如图3.9, 随着分块大小不断增大, 静态调度不受影响, 动态和引导调度的耗时均先下降而后上升, 在分块大小为 16 时达到最优。这是由于当分块变大时, 分配次数就减少, 线程切换和锁竞争的次数也就减少, 其开销便减小, 虽然线程单次获取到的任务量变大, 但总任务量不变, 无影响。而当分块继续变大时, 会导致负载不均, 则运行时间变长。

分块大小	串行	静态	动态	引导
0	127.87	35.114	42.886	37.822
1	127.87	34.537	43.126	37.279
2	127.87	34.877	39.032	36.636
4	127.87	35.446	37.345	35.702
8	127.87	34.627	35.439	34.602
16	127.87	34.334	34.613	34.383
32	127.87	34.65	35.906	34.947
64	127.87	34.841	37.814	37.476
128	127.87	35.394	42.134	42.657

表 7: 不同分块大小的各方法运行时间 (单位: ms)

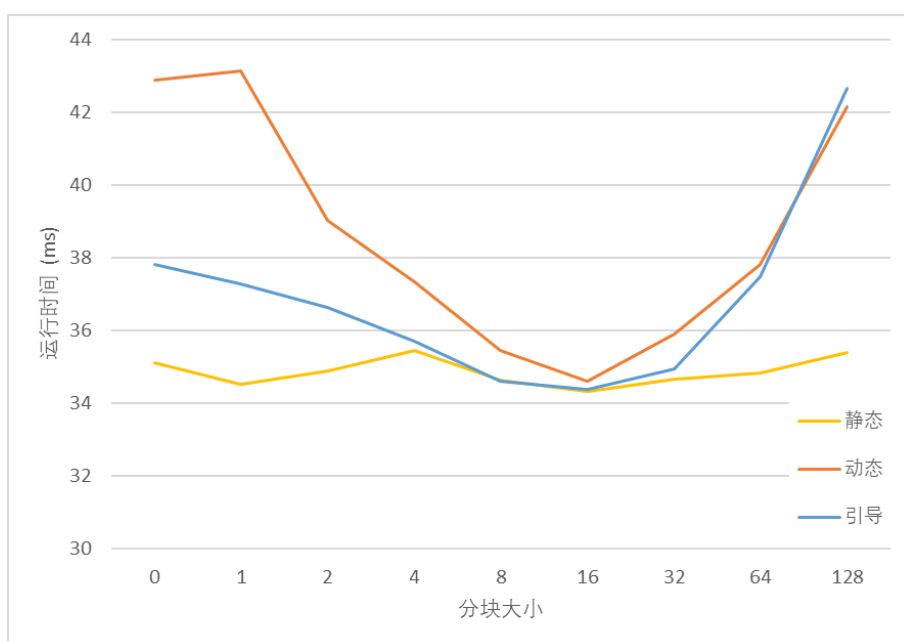


图 3.9: 不同分块大小的各方法运行时间

(3) 不同数据规模下各个方法的运行时间对比

如图3.10可以看到, 数据规模越大, 并行编程带来的收益也越大。

从图11(a)中又可看出, 并行编程并非始终占优, 当任务量较小时, 线程切换等开销反而会使其不如单纯的串行方法的效率。

从图11(b)中看出, 动态调度和引导调度都是优于静态调度的, 因为他们有效避免了负载不均。

数据规模	串行	静态	动态	引导
128	0.3381	3.7695	3.6617	3.6666
256	2.3188	7.0228	8.1057	7.8278
384	7.3293	10.692	13.543	12.025
512	18.24	15.036	21.273	18.798
640	35.655	21.983	28.345	25.329
768	59.846	31.867	37.396	31.102
896	94.762	44.252	41.28	29.48
1024	136.48	58.187	49.465	38.581
1152	197.83	72.57	56.061	49.005
1280	269.52	95.766	73.812	61.537
1408	355.3	118.16	89.132	74.395
1536	462.37	139.72	105.18	91.83
1664	586.4	181.27	126.82	113.75
1792	730.35	186.49	150.21	137.74
1920	893.24	251.89	211.42	188.38
2048	1076.7	288.03	278.65	235.69
2176	1314.1	303.69	291.62	276.27
2304	1519	378.61	365.43	309.05
2432	1770.4	379.95	405.7	358.65
2560	2061.6	561.03	466.14	419.51

表 8: 不同数据规模下各方法运行时间 (单位: ms)

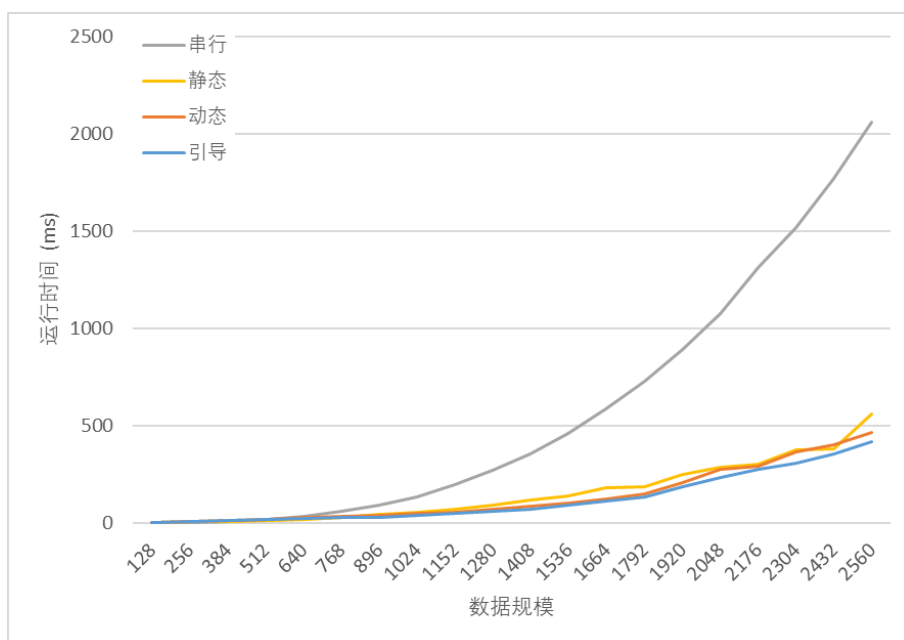
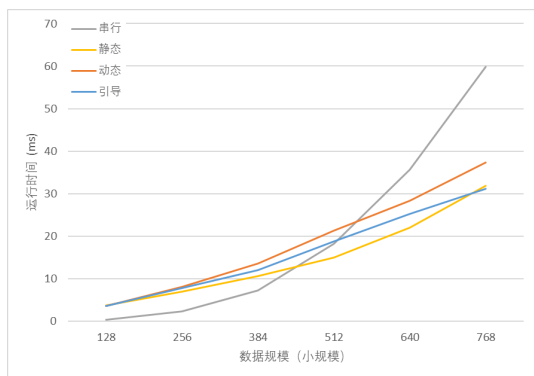
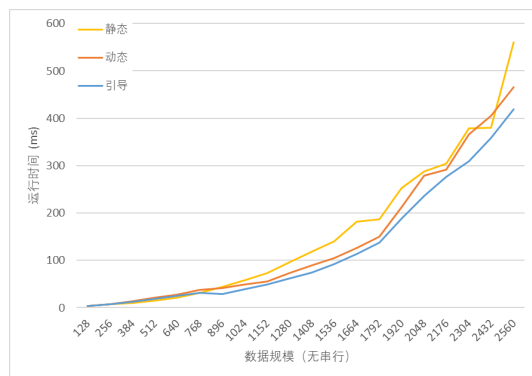


图 3.10: 不同数据规模下各方法运行时间



(a) 小规模



(b) 无串行

图 3.11: 不同数据规模下各方法运行时间 (细节)

3.3.3 实际运行结果

线程数	串行	静态	动态	引导
2	127.87	75.971	88.541	73.72
4	127.87	48.975	54.539	45.684
6	127.87	53.034	45.349	38.492
8	127.87	48.887	43.139	37.566
10	127.87	48.55	53.585	40.801
12	127.87	57.127	57.416	45.395
14	127.87	55.455	71.486	66.063
16	127.87	60.001	78.574	73.432
18	127.87	65.181	90.522	82.666
20	127.87	73.245	91.233	86.712
22	127.87	81.999	100.45	93.318
24	127.87	91.38	106.42	101.67
26	127.87	96.509	114.65	109.29
28	127.87	106.98	115.13	111.1
30	127.87	113.61	122.79	116.62
32	127.87	114.15	130.07	125.9
数据规模: 10000	线程数: 8			
分块大小	串行	静态	动态	引导
0	127.87	35.114	42.886	37.822
1	127.87	34.537	43.126	37.279
2	127.87	34.877	39.032	36.636
4	127.87	35.446	37.345	35.702
8	127.87	34.627	35.439	34.602
16	127.87	34.334	34.613	34.383
32	127.87	34.65	35.906	34.947
64	127.87	34.841	37.814	37.476
128	127.87	35.394	42.134	42.657
chunk: 0	线程数: 8			
数据规模	串行	静态	动态	引导
128	0.3381	3.7695	3.6617	3.6666
256	2.3188	7.0228	8.1057	7.8278
384	7.3293	10.692	13.543	12.025
512	18.24	15.036	21.273	18.798
640	35.655	21.983	28.345	25.329
768	59.846	31.867	37.396	31.102
896	94.762	44.252	41.28	29.48
1024	136.48	58.187	49.465	38.581
1152	197.83	72.57	56.061	49.005
1280	269.52	95.766	73.812	61.537
1408	355.3	118.16	89.132	74.395
1536	462.37	139.72	105.18	91.83
1664	586.4	181.27	126.82	113.75
1792	730.35	186.49	150.21	137.74
1920	893.24	251.89	211.42	188.38
2048	1076.7	288.03	278.65	235.69
2176	1314.1	303.69	291.62	276.27
2304	1519	378.61	365.43	309.05
2432	1770.4	379.95	405.7	358.65
2560	2061.6	561.03	466.14	419.51

图 3.12: 高斯消元法的 OpenMP 实现